1. 串行执行、并发执行、并行执行

串行执行、并发执行和并行执行是计算机程序执行方式的三个不同阶段，它们的发展历史与计算机硬件和软件的进步紧密相关。

1.1 串行执行

串行执行属于最早期的程序执行方式，是指任务按顺序一个接一个地执行。在串行执行中，一个任务必须等待前一个任务完成后才能开始执行。这种执行方式简单、易于控制，但效率较低，特别是在处理大量任务时，因为CPU的运算能力没有得到充分利用。在早期计算机系统中非常普遍，因为当时的硬件资源有限，且CPU的能力远不如现在，无法同时执行多个任务，并且大部分计算机都只有一个CPU。因此程序设计通常采用串行执行，在这一时期，计算机主要用于科学计算和军事目的，任务通常是简单的、顺序的，不需要复杂的并发或并行处理。

串行执行具有以下几个特点：

- 顺序性：任务必须按照一定的顺序执行，一个任务完成后才能开始下一个任务。
- 易于管理：由于任务按顺序执行，因此管理和调试相对简单。
- 资源利用率低：在执行单个任务时，CPU的运算能力没有得到充分利用，可能导致效率低下。

1.2 并发执行

随着计算机技术的发展和多用户需求的出现，单一串行执行方式逐渐显得力不从心。此时操作系统引入了进程的概念，且它允许多个程序在内存中同时存在，操作系统负责调度这些程序，通过时间片轮转等技术使得它们可以交替使用CPU，从而达到一种看似并发的效果。这一技术提高了CPU的利用率，因为当一个程序等待I/O操作时，CPU可以切换到另一个程序执行。这一时期，计算机开始被广泛应用于商业和科学领域，需要处理更多的数据和更复杂的任务。

并发执行具有以下几个特点：

- **效率提高：**通过在不同的任务之间切换，提高了CPU的利用率。
- **响应性增强：**在交互式应用中，可以提高用户体验，比如用户在等待某个计算密集型任务的同时还可以进行其他操作。
- **复杂性增加：**并发编程需要处理诸如同步、通信等问题，增加了开发难度。

华清远见HQYJ宁波实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

1.3 并行执行

随着晶体管和集成电路技术的进步，计算机的处理器能力得到了显著提升，并且制造出了包含多个处理器的计算机系统，这促进了并行计算的发展。并行执行能够显著提高计算机处理大量数据和复杂计算的能力，特别是在科学研究、天气预报、数据分析等领域。随着时间的发展，并行计算逐渐成为高性能计算（HPC）的核心技术，并行处理技术也在多核处理器和分布式系统中得到了广泛应用。

并行执行具有以下几个特点：

- **高性能：**通过将任务分解为多个子任务并分配给不同的处理器执行，可以显著提高执行速度。
- **复杂性高：**并行编程通常需要对算法进行重新设计以支持并行化，同时还需要考虑数据共享、同步等问题。
- **可扩展性：**并行系统的设计往往考虑到了将来扩展的可能性，可以通过增加处理器数量来提高性能。

1.3 例子

假设有两个任务A和B，以下是串行执行和并发执行的对比：

- **串行执行**：
  1. 开始执行任务A
  2. 任务A完成
  3. 开始执行任务B
  4. 任务B完成
- **并发执行**：
  1. 开始执行任务A
  2. 同时（或者在任务A执行了一部分后）开始执行任务B
  3. 任务A和任务B可能同时完成，或者一个在另一个之前完成

在实际应用中，串行执行和并发执行的选择取决于任务的性质、系统的能力和性能要求。对于IO密集型任务，时分并发执行可以提升用户体验，因为一个任务在等待IO操作时，CPU可以切换到另一个任务。对于CPU密集型任务，并行并发执行可以显著提高程序的执行速度。

2. I/O密集型任务与CPU密集型任务

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

IO密集型任务（I/O-bound tasks）和CPU密集型任务（CPU-bound tasks）是根据任务执行时主要消耗的资源类型来分类的。

2.1 I/O密集型任务

IO密集型任务是指那些任务执行过程中，大部分时间花费在输入/输出操作上，而不是CPU计算。这些任务的性能瓶颈通常在于磁盘读写、网络通信或用户输入等IO操作的速度。以下是IO密集型任务的特点：

- **等待时间**：任务大部分时间在等待IO操作完成。
- **CPU使用率**：CPU使用率通常不高，因为CPU在等待IO操作完成期间常常处于空闲状态。
- **并发优势**：IO密集型任务可以通过并发执行来提高效率，因为在等待一个IO操作完成时，CPU可以切换到另一个任务执行。

例子包括：文件处理、网络请求处理、数据库操作等。

2.2 CPU密集型任务

CPU密集型任务是指那些任务执行过程中，大部分时间用于CPU计算，涉及大量的算术运算、逻辑判断和数据处理。这些任务的性能瓶颈通常在于CPU的计算能力。以下是CPU密集型任务的特点：

- **计算时间**：任务大部分时间在进行计算。
- **CPU使用率**：CPU使用率通常很高，因为任务需要大量的CPU资源。
- **并行优势**：CPU密集型任务可以通过并行执行在不同的处理器核心上同时运行来提高性能。

例子包括：视频编码、图像处理、科学计算等。

2.3 对程序执行的影响

- 对于IO密集型任务，多线程通常是一个较好的选择，因为它可以在单个进程内有效地处理多个IO操作，而不会因为进程间切换的开销而降低效率。
- 对于CPU密集型任务，多进程可以更好地利用多核CPU的能力，因为每个进程可以在不同的核心上并行执行，从而提高整体的处理速度。

3. 单核CPU与多核CPU

3.1 单核CPU

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- **核心数量**：单核CPU只有一个处理器核心。
- **任务处理**：在执行多任务时，单核CPU通过时分共享（time-sharing）的方式在任务之间快速切换，给人一种同时执行的错觉，但实际上在任何给定时刻只有一个任务在执行。
- **性能限制**：由于只有一个核心，单核CPU的性能受限于单个核心的处理能力，无法真正实现并行处理。

3.2 多核CPU

- **核心数量**：多核CPU拥有两个或更多的处理器核心，每个核心都可以独立执行任务。
- **任务处理**：多核CPU可以真正并行地处理多个任务或线程，每个核心可以同时执行一个任务，从而显著提高整体的处理能力。
- **性能潜力**：多核CPU的性能潜力更大，特别是在执行多任务应用程序时，能够提供更高的吞吐量和更快的响应时间。

4. 多进程的概念

4.1 进程介绍

进程：进程就是程序的一次执行过程，就是一个正在执行的任务，一段程序的每一次运行都会产生一个或多个进程。进程是有生命周期的，大部分会随着程序的运行而创建，随着程序的结束而终止，也可以去手动结束进程。在操作系统中，进程是操作系统进行资源分配和调度的基本单位。每个进程都有自己的私有地址空间、执行堆栈、程序计数器、局部变量以及其他系统资源（如文件描述符、网络连接等）等。通俗的说，一个正在运行的程序就是一个进程，比如QQ、微信等，但也有可能这个程序会生成多个进程。

4.2 进程和程序的关系

进程：进程是程序的一次执行过程，它是动态的，具备生命周期，在程序运行时存在，程序执行完毕及用户主动结束、系统错误等都会导致进程结束。

程序：程序是静态的，没有生命周期，在磁盘上存放，由一系列指令和数据组成的文件，这些文件可以被操作系统加载到内存中并执行。

4.3 进程的种类

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

在Windows操作系统中，进程可以根据其创建方式和执行环境被分为几种类型。以下是一些常见的Windows进程类型：

1. **系统进程（System Process）**
   - 这些进程是操作系统启动时由系统本身创建的，它们负责管理系统的核心功能，如内存管理、设备驱动程序、安全性和其他系统级服务。
   - 例如：lsass.exe（本地安全认证子系统服务）、wininit.exe(Windows初始化进程)。
2. **服务进程（Service Process）**
   - 这些进程通常在系统启动时或按需启动，它们在后台运行，提供网络、安全、系统维护等服务，通常不直接与用户交互。
   - 例如：`DHCP Client`（管理IP地址分配）。
3. **用户进程（User Process）**
   - 这些是由用户启动的应用程序创建的进程，用于执行特定任务，如文字处理、网页浏览、游戏等。
   - 例如：`winword.exe`（Microsoft Word）、`chrome.exe`（Google Chrome浏览器）。
4. **交互式进程（Interactive Process）**
   - 这些是与用户交互的进程，它们通常在用户登录并直接与操作系统交互时运行。
   - 例如：命令提示符（`cmd.exe`）或PowerShell（`powershell.exe`）。

除此之外，还有一些特殊进程：

1. **守护进程（Daemo Process）**

   守护进程（Daemon）是一种在Unix和类Unix操作系统中运行的后台进程。它们通常在系统启动时自动启动，并在后台执行特定的系统级任务，不需要用户直接干预。在Windows操作系统中，没有与Unix和类Unix系统中的守护进程（Daemon）完全等同的概念。然而，Windows有类似的系统服务，它们在功能上与守护进程相似，都是在后台运行，执行系统级任务，通常不需要用户直接干预。

2. **僵尸进程（Zombie Process）**

   僵尸进程是指那些已经结束但仍然存在于进程表中，等待父进程收集其退出状态的进程。它们不再占用系统资源，除了进程表中的一个条目，以及一个退出状态码。僵尸进程通常不会占用CPU时间，但它们仍然存在于进程表中，直到父进程去收集其退出状态。在Windows中这种情况不常见，因为进程在Windows中结束后会通过特定的机制去处理子进程的终止状态。

华清远见元宇宙实验中心 yyzlab.com.cn

3. **孤儿进程（Orphan Process）**

   孤儿进程是指那些父进程已经结束，而它们仍然在运行的进程。由于孤儿进程的父进程已经终止，它们会被init进程（进程号为1）所领养。init进程负责处理这些孤儿进程，确保它们能够正常结束。而在Windows中，孤儿进程会被系统进程收养，确保他们能继续运行或正常结束。

4.4 PID

PID是“进程标识符”（Process Identifier）的缩写，它是一个由操作系统分配给每个进程的唯一数字（大于等于0）。在操作系统中，每个进程都会被赋予一个唯一的PID，以便系统可以追踪和管理这些进程。

1. **唯一性**：在一个操作系统中，每个正在运行的进程都有一个唯一的PID。即使在进程结束后，该PID通常也不会立即被重新分配给其他进程，以避免混淆。
2. **进程管理**：操作系统使用PID来识别和管理进程。例如，可以使用PID来发送信号给进程（如终止信号）、检查进程的状态、或者调整进程的优先级。
3. **系统资源**：PID还用于关联进程使用的系统资源，如打开的文件、网络连接和内存分配。
4. **父进程与子进程**：每个进程除了有自己的PID外，还有一个与之关联的父PID（PPID）。父PID是指启动该进程的进程的PID。
5. **工具和命令**：在命令行界面中，可以使用各种工具和命令来查看和管理进程，如 `ps`（Unix-like系统）、`tasklist`（Windows）等，这些工具通常会显示进程的PID。

kill命令：

在Windows中，可以在CMD中通过kill + PID 来终止该进程的运行。

在Linux中，kill命令会更加强大。

4.5 多进程的概念

多进程（Multiprocessing）是指在同一时间内同时执行多个程序的技术或能力，每个进程都有自己的内存空间、文件描述符及其他系统资源。

以下是多进程的一些特点：

1. **并发执行**：多进程可以在多核或多处理器系统上实现真正的并行执行，即不同的进程可以在不同的CPU核心上同时运行。
2. **资源分配**：每个进程通常拥有独立的内存空间，这意味着它们不共享内存，这减少了资源共享带来的复杂性和潜在的问题。

华清远见云计算实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

3. **独立性**：全局变量在多个进程中不共享资源，进程之间的数据是独立的，默认是互不影响的。且由于进程间相对独立，一个进程的失败通常不会影响到其他进程，这提高了系统的稳定性和可靠性。
4. **进程间通信（IPC）**：进程间通信机制（如管道、消息队列、共享内存、信号量等）用于允许进程之间交换数据和同步操作。

5. Python中创建多进程

在Python中，**multiprocessing**库提供了创建和管理进程的方法，它允许程序员创建进程，并提供了一系列的API来支持进程间数据共享、同步和通信。

在Python中，使用multiprocessing中的Process类来创建子进程，Process类创建对象时的参数为：

```
multiprocessing.Process(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None)
```

以下是各个参数的说明：

- `group`：通常不使用，是为将来可能的扩展预留的。
- `target`：表示调用对象，即子进程要执行的任务。这个参数通常是一个函数的名字。
- `name`：进程的名称。默认情况下，进程名称为`Process-N`，其中`N`是进程的序号。可以通过这个参数来指定一个自定义的名称。
- `args`：表示调用`target`函数时传递的参数元组。
- `kwargs`：表示调用`target`函数时传递的参数字典。
- `daemon`：如果设置为`True`，则子进程将是一个守护进程。当主进程结束时，所有守护进程都将被终止。默认值为`None`，表示继承当前进程的守护进程设置。

创建的对象也拥有一些方法，分别是：

1. `start()`：启动进程。这将执行在创建`Process`对象时指定的`target`函数，调用start时会自动调用run。
2. `run()`：此方法用于定义进程启动时执行的操作。默认情况下，它调用传递给`target`参数的函数。如果`target`没有指定，你可以覆盖此方法来实现自定义的进程行为。
3. `join([timeout])`：主进程等待子进程终止或直到达到指定的超时时间。如果`timeout`被省略或为`None`，则主进程将一直停留在这里。
4. `is_alive()`：返回一个布尔值，表示进程是否仍然在运行。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

5. `terminate()`：强行终止进程，且不会进行任何清理操作。 如果该进程创建了子进程，那么这个子进程就变成了僵尸进程，如果p还保存了一个锁，那么这个锁也不会被释放，会变成死锁。
6. `kill()`：终止进程。在Unix上，这是通过发送SIGKILL信号实现的；在Windows上，则是通过调用`TerminateProcess`。
7. `close()`：关闭进程。此方法释放Process对象所持有的资源，如果子进程仍在进行，调用此方法将是错误的。

除此之外，还有属性：

1. name：返回或设置进程的名称。
2. daemon：返回或设置进程是否为守护进程，如果设置的话，必须在start()之前设置。
3. pid：返回进程的PID。
4. exitcode：返回进程的退出代码，如果进程未结束，就返回None，负值-N表示子进程被信号N终止，正常终止返回0。
5. authkey：返回或设置进程间通信的密钥，用于进程间通信的身份认证，了解即可。

注意：Windows中的Process必须放到 if __name__ == "__main__"下面。

5.1 创建不传参的进程

5.1.1 查看各进程的PID及PPID

```
import os
from multiprocessing import Process


def func1():
    my_pid = os.getpid()
    ppid = os.getppid()
    print(f'执行func2的pid是：{my_pid}')
    print(f'执行func2的ppid是：{ppid}')


def func2():
    my_pid = os.getpid()
    ppid = os.getppid()
    print(f'执行func1的pid是：{my_pid}')
```

华清远见嵌入式实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    print(f'执行func1的ppid是：{ppid}')


if __name__ == '__main__':
    p1 = Process(target=func1)
    p1.start()

    p2 = Process(target=func2)
    p2.start()

    # 获取当前进程的PID
    current_pid = os.getpid()
    print(f"主进程的PID为: {current_pid}")
    p1.join()
    p2.join()
```

5.1.2 运行时间统计

```
import time
from multiprocessing import Process


def func1():
    print("Function 1 is running.")
    time.sleep(2)
    print("Function 1 has finished.")


def func2():
    print("Function 2 is running.")
    time.sleep(2)
    print("Function 2 has finished.")


if __name__ == '__main__':
    start_time = time.time()  # 记录程序开始时间

    # 创建一个子进程来运行func2
    process = Process(target=func2)
    process.start()
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    # 获取创建的进程的PID
    pid = process.pid

    # 主进程运行func1
    func1()

    # 等待子进程结束
    process.join()

    # 计算并打印整个程序的运行时间
    total_time = time.time() - start_time
    print(f"Total execution time: {total_time:.2f} seconds")
```

5.2 创建传参的进程

5.2.1 使用args传参

```
from multiprocessing import Process
import time


def say_hello(name):
    time.sleep(2)
    print(f'Hello {name}, Nice to meet you!')


if __name__ == '__main__':
    start = time.time()
    process1 = Process(target=say_hello, args=('Alice', ))
    process1.start()
    say_hello('Bob')
    process1.join()
    exe_time = time.time() - start
    print(exe_time)
```

5.2.2 使用kwargs传参

```
from multiprocessing import Process
import time
```

华清远见元宇宙实验中心 yyzlab.com.cn

```
def say_hello(name):
    time.sleep(2)
    print(f'Hello {name}, Nice to meet you!')


if __name__ == '__main__':
    start = time.time()
    process1 = Process(target=say_hello, kwargs={'name': 'Alice'})
    process1.start()
    say_hello('Bob')
    process1.join()
    exe_time = time.time() - start
    print(exe_time)
```

5.3 一次性创建多个进程

```
import sys
from multiprocessing import Process
import time


def say_hello(name):
    time.sleep(1)
    print(f'Hello {name}, Nice to meet you!')
    # 刷新标准输出缓冲区
    sys.stdout.flush()

if __name__ == '__main__':
    names = ['Zhangsan', 'Lisi', 'Wangwu', 'Zhaoliu', 'Tianqi']
    processes = []
    start = time.time()
    for i in range(len(names)):
        process = Process(target=say_hello, args=(names[i], ))
        process.start()
        processes.append(process)

    for process in processes:
        process.join()

    exe_time = time.time() - start
    print(exe_time)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

6. 进程间通信

进程间通信（Inter-Process Communication, IPC）是指在不同进程之间传送数据或信号的一些方法。在多进程程序中，IPC是非常重要的，因为它允许进程协同工作、共享数据或同步操作。以下是几种常见的进程间通信机制：

6.1 管道（pipe）

管道是一种在父子进程间或兄弟进程间进行通信的机制。Python的`multiprocessing`模块提供了`Pipe()`函数，可以用来创建管道。

6.1.1 创建管道

```
parent_conn, child_conn = Pipe()
```

使用`multiprocessing.Pipe()`可以创建一个管道。这个函数返回一个由两个连接对象组成的元组，这两个对象分别代表管道的两端。默认情况下，管道是双向的，每个端点都可以即读又写。

```
from multiprocessing import Process, Pipe

# 创建一个管道
parent_conn, child_conn = Pipe()
```

6.1.2 管道方法

在上面的例子中，parent_conn和child_conn都是管道对象，它们都拥有共同的方法：

- **send(obj)**：发送一个对象到管道的另一端。这个对象必须是可序列化的。
- **recv()**：从管道的另一端接收一个对象。该方法是阻塞的。
- **close()**：关闭管道连接。当不再需要管道时，应该调用这个方法来释放资源。
- **fileno()**：返回由连接对象使用的文件描述符。
- **poll(timeout)**：返回连接对象中是否有可以读取的数据。如果未指定timeout，会马上返回，如果timeout是一个数字，则指定了阻塞的最大秒数，如果未指定timeout，那么将一直等待。
- **send_bytes(buffer, offset, size)**：通过连接发送buffer，offset是buffer中的偏移量，size是要发送的字节数。数据以一条完整的数据发送。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- **recv_bytes(maxlength)**：以字符串的形式返回一条从连接对象另一端发送过来的字节数据。此方法在接收到数据前一直阻塞。如果连接对象被关闭或没有数据可读取，将抛出异常。如果消息长度大于maxlength，则会抛出异常且该连接对象不可再读。
- **recv_bytes_into(buffer, offset)**：将一条完整的数据读入buffer中并返回消息的字节数，此方法在接收到数据前一直阻塞。如果连接对象被关闭或没有数据可读取，将抛出异常。offset指定buffer中放置消息处的字节偏移量。如果消息长度大于buffer将抛出异常。

6.1.3 在进程中使用管道

管道可以在进程间传递数据。通常，一个连接会传递给一个子进程，而另一个连接保留在父进程中。

```
from multiprocessing import Process, Pipe


def child_process(conn):
    # 子进程从连接中接收数据
    conn.send('Hello from child')
    data = conn.recv()
    print('Received:', data)
    conn.close()


if __name__ == '__main__':
    # 创建一个管道
    parent_conn, child_conn = Pipe()
    # 创建子进程
    p = Process(target=child_process, args=(child_conn,))
    p.start()

    # 父进程发送数据到子进程
    parent_conn.send('Hello from parent')
    print('Received:', parent_conn.recv())

    # 等待子进程结束
    p.join()
```

```
import multiprocessing
import time
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
def send_data(pipe):
    # 发送数据前先等待一会儿，以便观察recv方法的阻塞行为
    time.sleep(2)
    pipe.send('Hello from parent')


def receive_data(pipe):
    print("Child is waiting to receive data...")
    # 尝试从管道接收数据，这个调用是阻塞的
    data = pipe.recv()
    print(f"Child received: {data}")

if __name__ == '__main__':
    # 创建一个管道
    parent_pipe, child_pipe = multiprocessing.Pipe()

    # 创建子进程
    child_process = multiprocessing.Process(target=receive_data,
args=(child_pipe,))
    child_process.start()

    # 在父进程中发送数据
    send_data(parent_pipe)

    # 等待子进程结束
    child_process.join()
```

华清远见元宇宙实验中心 yyzlab.com.cn

6.1.4 管道特点

- **双向通信**：管道允许两个方向的通信，即每个管道有一个接收端和一个发送端。
- **点对点连接**：管道通常用于两个进程之间的直接通信，不支持多个进程之间的通信。
- **管道大小有限**：管道的缓冲区大小是有限的。如果缓冲区满了，发送操作会阻塞。

6.1.5 注意事项

- 管道默认是双向的，但也可以通过设置`duplex=False`来创建单向管道。此时返回的第一个对象只能接收消息，第二个对象只能发送消息。
- 当使用管道在进程间传递大量数据时，要注意管道可能会成为性能瓶颈。

华清远见 HQYJ.COM

```python
# 单向管道
from multiprocessing import Process, Pipe


def sender(conn):
    conn.send([42, None, 'hello'])
    conn.close()


if __name__ == '__main__':
    # 创建单向管道，parten_conn是只读的，child_conn是只写的
    parent_conn, child_conn = Pipe(duplex=False)

    # 创建发送者进程
    # sender函数应该使用child_conn，因为它是可写的
    sender_process = Process(target=sender, args=(child_conn,))
    # 启动进程
    sender_process.start()

    print(parent_conn.recv())

    # 等待进程结束
    sender_process.join()
```

6.2 消息队列（Queue）

消息队列提供了一种在进程间传输数据的方式，这种方式是通过在内核中维护一个消息队列来实现的。进程可以发送数据到队列，也可以从队列中接收数据。在Python的`multiprocessing`模块中，`Queue`类提供了一个先进先出（FIFO）的消息队列。

6.2.1 创建消息队列

```python
from multiprocessing import Queue

# 创建一个消息队列
queue = Queue(maxsize=10)  # maxsize为队列中最多可以存放的元素数量
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

6.2.2 消息队列的方法

在上面的例子中，queue是创建出来的消息队列，它拥有下面几种方法：

- `put(obj, block=True, timeout=None)`：将obj放入队列，如果可选参数block是True而且timeout是None，将会阻塞当前进程，直到有空的缓冲槽。如果timeout是正数，将会在阻塞了最多timeout秒之后还是没有可用的缓冲槽时抛出异常。如果block是False，那么在没有空的缓冲槽时，会立即抛出异常，此时timeout会被忽略。
- `get(block=True, timeout=None)`：从消息队列里获取消息。该方法为阻塞等待的方法。block和timeout的作用与put一致。
- `empty()`：如果队列为空，返回`True`，否则返回`False`。
- `full()`：如果队列满了，返回`True`，否则返回`False`。
- `qsize()`：返回队列中当前元素的数量。
- `get_nowait()`：立即尝试从队列里获取一个元素，如果队列为空，抛出`Queue.Empty`异常。
- `put_nowait()`：立即尝试向队列里放入一个元素，如果队列满了，抛出`Queue.Full`异常。

6.2.3 在进程中使用消息队列

```
from multiprocessing import Process, Queue
import time


def process1(process_queue):
    print('准备接收数据')
    print('接收到的数据为: ', process_queue.get())


if __name__ == '__main__':
    process_queue = Queue(5)
    p1 = Process(target=process1, args=(process_queue, ))
    p1.start()
    time.sleep(2)
    process_queue.put('hello')
    p1.join()
    p1.close()
```

华清远见 yyzlab.com.cn

华清远见 HQYJ.COM

6.2.4 消息队列的特点

- **先进先出（FIFO）**：队列遵循先进先出（FIFO）的原则，即先放入队列的元素会先被取出。
- **同步访问**：`Queue`类提供了一系列同步方法，如`put()`、`get()`等，以确保多进程对队列的访问是安全的。
- **容量限制**：队列可以指定最大容量，当队列满时，新元素将无法放入；当队列空时，试图从队列中获取元素的进程将阻塞，直到有新元素放入队列。
- **生产者-消费者模式**：`Queue`类非常适合用于生产者-消费者模式，其中生产者进程将数据放入队列，而消费者进程从队列中取出数据。

```python
from multiprocessing import Process, Queue
import time


# 生产者函数
def producer(queue):
    for i in range(5):
        queue.put(f"Product {i}")
        print(f"Produced {i}")
        time.sleep(1)


# 消费者函数
def consumer(queue):
    while True:
        product = queue.get()
        if product is None:
            break
        print(f"Consumed {product}")


if __name__ == '__main__':
    queue = Queue(5)
    p = Process(target=producer, args=(queue,))
    c = Process(target=consumer, args=(queue,))

    p.start()
    c.start()

    p.join()
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    queue.put(None)  # 发送结束信号给消费者
    c.join()
```

6.2.4 注意事项

- **避免全局共享**: 不要在多个进程间共享同一个队列实例，而应该为每个进程创建单独的队列实例。
- **设置队列大小**: 如果没有指定队列的大小，它将默认为无限大小。这可能导致内存问题，特别是当生产者产生消息的速度远大于消费者消费消息的速度时。
- **处理队列异常**: 当队列操作失败时（如队列已满或为空），应当捕获并处理相应的异常。
- **队列性能**: 队列操作可能会影响性能，尤其是在高并发环境下。需要根据应用需求选择合适的队列实现和大小。

6.3 共享内存

共享内存是一种进程间通信（IPC）机制，顾名思义，它允许多个进程访问同一块内存空间。每个进程都可以读取或写入这块内存，从而实现数据的共享。

6.3.1 创建共享内存

在Python中，共享内存分为两种，一种是共享一个变量，一种是共享一个数组。

1. 在Python中，使用`multiprocessing.Value(type_code, *args, lock=True)`来创建一个共享变量，其中type_code表示类型代码，*args表示初始化变量的值。lock表示锁，默认会创建一个锁用来保护共享变量。如果传入False，Value的实例就不会被锁保护，它将不是进程安全的。
2. 在Python中，使用`multiprocessing.Array(type_code, size_or_initializer, lock=True)`来创建一个共享数组，其中type_code表示类型代码，size_or_initializer表示数组的大小或初始化值，如果是一个整数，则表示数组的长度，且数组将被初始化为0，如果是一组序列，则就是数组的初始化值，其长度决定数组的长度。lock表示锁，默认会创建一个锁用来保护共享数组。

华清远见嵌入式学习实验中心 www.fvzlab.com.cn

华清远见 HQYJ.COM

类型代码	映射	描述
'b'	ctypes.c_byte	有符号字节（8位）
'B'	ctypes.c_ubyte	无符号字节（8位）
'h'	ctypes.c_short	有符号短整型（16位）
'H'	ctypes.c_ushort	无符号短整型（16位）
'i'	ctypes.c_int	有符号整型（通常是32位，但依赖于平台）
'I'	ctypes.c_uint	无符号整型（通常是32位，但依赖于平台）
'l'	ctypes.c_long	有符号长整型（32位）
'L'	ctypes.c_ulong	无符号长整型（32位）
'q'	ctypes.c_longlong	有符号长长整型（64位）
'Q'	ctypes.c_ulonglong	无符号长长整型（64位）
'f'	ctypes.c_float	单精度浮点数（32位）
'd'	ctypes.c_double	双精度浮点数（64位）

```
from multiprocessing import Value, Array

shared_num = Value('i', 0)
shared_array = Array('i', range(10))
```

6.3.2 共享内存的方法

- `value`：对于`Value`对象，`value`属性用于获取或设置共享变量的值。
- `[:]`：对于`Array`对象，可以使用切片操作来获取或修改数组中的元素。

6.3.3 在进程中使用共享内存

```
from multiprocessing import Process, Value, Array


def func(shared_num, shared_array):
    shared_num.value += 1
    for i in range(len(shared_array)):
        shared_array[i] += 1
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
if __name__ == '__main__':
    shared_num = Value('i', 0)
    shared_array = Array('i', range(10))

    p = Process(target=func, args=(shared_num, shared_array))
    p.start()
    p.join()

    print(shared_num.value)  # 输出 1
    print(shared_array[:])   # 输出 [1, 2, 3, ..., 10]
```

6.3.4 共享内存的特点

- **高效的数据共享**：共享内存比其他IPC机制（如消息队列）更高效，因为它避免了数据的复制。
- **同步问题**：共享内存需要同步机制（如锁）来防止竞态条件。
- **类型限制**：共享内存的数据类型有限，通常只能是基本数据类型。

6.3.5 注意事项

- **数据同步**：在使用共享内存时，应该使用锁（`Lock`或`RLock`）来同步对共享数据的访问，以避免竞态条件。
- **避免死锁**：在使用锁时，要注意正确的加锁和解锁顺序，以避免死锁。
- **内存管理**：共享内存不会自动清理，需要确保所有进程完成对共享内存的操作后，适当地关闭或释放内存。
- **安全性**：共享内存的使用需要谨慎，因为它可能会被未经授权的进程访问。
- **初始化值**：在创建共享内存时，应该提供正确的初始化值，以确保数据的一致性。

7. 进程同步

在Python中，进程同步是指在多进程环境下协调各个进程对共享资源的访问，主要解决的问题是当多个进程并发访问共享资源时，如何确保任意时刻只有一个进程能够访问该资源，从而避免由于进程间的无序竞争而导致的系统资源冲突，确保系统的稳定运行。进程同步通常涉及到以下几个核心概念：

1. **临界资源**是指一段时间内仅允许一个进程访问的资源，这可能是硬件资源，也可能是软件资源如变量、数据、表格、队列等。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

2. **临界区**是指访问临界资源的那部分代码。在进入临界区之前，需要检查是否可以访问临界资源，以确保资源的互斥访问。

进程同步的机制应遵循以下规则：

1. **空则让进**：如果临界资源处于空闲状态，则进程可以进入其临界区。
2. **忙则等待**：如果临界资源正在被使用，则请求访问的进程需要等待。

常见的进程同步机制包括锁、信号量、事件、条件变量等。

7.1 锁

锁（Lock）是一种用于控制多个进程访问共享资源的机制，锁的主要目的是防止多个进程同时访问共享资源时可能产生的竞态条件（Race Condition），确保数据的一致性和完整性。在Python中，最常用的锁为互斥锁（Lock）和递归锁（RLock）。

7.1.1 互斥锁

这是最常见的一种锁，它确保同一时间只有一个进程可以访问共享资源。当一个进程正在使用资源时，它会锁定该资源，其他进程必须等待锁被释放后才能访问。在`multiprocessing`模块中，`Lock`对象可以用来确保临界区代码的互斥执行。

方法：

- `acquire(blocking=True, timeout=-1)`：尝试获取锁。如果`blocking`为`True`并且`timeout`是默认值`-1`，该方法会阻塞直到锁被获取。如果`blocking`为`False`，则立即返回而不阻塞。
- `release()`：释放锁。

```
import multiprocessing
from multiprocessing import Queue, current_process
import time


# 定义一个函数task，它将用于创建进程
def task(lock, queue, amount):
    # 循环确保进程能够重复执行任务
    while True:
        # 获取锁，确保在修改共享资源时不会被其他进程打断
        lock.acquire()  # 上锁

        # 从队列中获取当前的余额
```

华清远见 yyzlab.com.cn

华清远见 HQYJ.COM

```
        money = queue.get()

        # 检查余额是否足够进行取款
        if money >= amount:
            # 如果余额足够，进行取款操作
            money -= amount
            # 打印取款成功的信息
            print(f"{current_process().name}取了{amount}，现在还有
{money}")
        else:
            # 如果余额不足，打印取款失败的信息
            print(f"{current_process().name}取钱失败，银行没钱了！！")
            # 释放锁，允许其他进程继续操作
            lock.release()  # 解锁
            # 将余额放回队列，以便其他进程可以继续使用
            queue.put(money)
            # 退出循环，因为取款失败，不需要继续尝试
            break

        # 将更新后的余额放回队列
        queue.put(money)

        # 休眠一段时间，模拟取款操作需要的时间
        time.sleep(1)

        # 释放锁，允许其他进程继续操作
        lock.release()  # 解锁


# 主程序入口
if __name__ == '__main__':
    # 初始余额
    count = 1000
    # 创建一个队列，用于进程间通信
    queue = Queue(5)
    # 将初始余额放入队列
    queue.put(count)

    # 定义互斥锁，用于保护共享资源
    lock = multiprocessing.Lock()

    # 创建两个进程，分别模拟张三和李四的取款操作
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    t1 = multiprocessing.Process(target=task, args=(lock, queue, 50), name='张三')
    t2 = multiprocessing.Process(target=task, args=(lock, queue, 100), name='李四')

    # 启动两个进程
    t1.start()
    t2.start()

    # 等待两个进程完成
    t1.join()
    t2.join()
```

```
from multiprocessing import Lock, Queue, Process, current_process
import time


# 定义生产者函数，它会不断地向队列中添加数据
def producer(queue):
    while True:
        # 循环5次，每次向队列中添加'hello'
        for i in range(5):
            queue.put('hello')
        # 生产者进程休眠2秒
        time.sleep(2)


# 定义消费者函数，它会不断地从队列中取出数据
def consumer(queue, lock):
    while True:
        # 获取锁，确保在消费数据时不会被其他消费者进程打断
        lock.acquire()
        # 消费者进程休眠2秒
        # time.sleep(2)
        # 检查队列是否为空
        if not queue.empty():
            # 如果队列不为空，取出一个数据
            res = queue.get()
            # 打印取出的数据
            print(f'{current_process().name}: {res}')
        # 释放锁，允许其他消费者进程继续操作
```

华清远见云宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        lock.release()


# 主程序入口
if __name__ == '__main__':
    # 创建一个队列，用于生产者和消费者之间传递数据
    queue = Queue(10)
    # 创建一个互斥锁，用于保护队列
    lock = Lock()

    # 创建生产者进程
    producer_process = Process(target=producer, args=(queue,))
    producer_process.start()

    # 创建多个消费者进程
    consumer1_process = Process(target=consumer, args=(queue,
lock), name='consumer1')
    consumer1_process.start()

    consumer2_process = Process(target=consumer, args=(queue,
lock), name='consumer2')
    consumer2_process.start()

    consumer3_process = Process(target=consumer, args=(queue,
lock), name='consumer3')
    consumer3_process.start()

    consumer4_process = Process(target=consumer, args=(queue,
lock), name='consumer4')
    consumer4_process.start()

    consumer5_process = Process(target=consumer, args=(queue,
lock), name='consumer5')
    consumer5_process.start()

    # 等待生产者进程完成
    producer_process.join()

    # 等待所有消费者进程完成
    consumer1_process.join()
    consumer2_process.join()
    consumer3_process.join()
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    consumer4_process.join()
    consumer5_process.join()
```

除此之外，要避免死锁的出现。在Python中，多进程的死锁（Deadlock）是指在多个进程之间，每个进程都在等待其他进程释放资源，但是这些资源又被其他进程持有，导致所有进程都无法继续执行，形成了一种僵持状态。简单来说，死锁是多个进程因竞争资源而造成的一种互相等待的僵局。死锁通常发生在以下四个条件同时满足时：

1. **互斥条件**：资源不能被多个进程同时使用或不想被多个进程同时访问。
2. **占有和等待条件**：进程至少持有一个资源，并且正在等待获取额外的资源，而该资源又被其他进程持有。
3. **不可抢占条件**：已经分配给进程的资源在该进程完成任务前不能被抢占。
4. **循环等待条件**：存在一种进程资源的循环等待链，每个进程至少持有一个资源，并等待获取下一个进程所持有的资源。

避免死锁的出现，可以使用with语句，通过with来帮我们自动释放锁，从而可以大幅度减少死锁出现的可能。

```
from multiprocessing import Lock, Queue, Process, current_process
import time


# 定义生产者函数，它会不断地向队列中添加数据
def producer(queue):
    while True:
        # 循环5次，每次向队列中添加'hello'
        for i in range(6):
            queue.put('hello')
        # 生产者进程休眠2秒
        time.sleep(2)


# 定义消费者函数，它会不断地从队列中取出数据
def consumer(queue, lock):
    while True:
        # 获取锁，确保在消费数据时不会被其他消费者进程打断，并且在执行完毕后自动释放锁
        with lock:
            # 消费者进程休眠2秒
            # time.sleep(2)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
            # 检查队列是否为空
            if not queue.empty():
                # 如果队列不为空，取出一个数据
                res = queue.get()
                # 打印取出的数据
                print(f'{current_process().name}: {res}')


# 主程序入口
if __name__ == '__main__':
    # 创建一个队列，用于生产者和消费者之间传递数据
    queue = Queue(10)
    # 创建一个互斥锁，用于保护队列
    lock = Lock()

    # 创建生产者进程
    producer_process = Process(target=producer, args=(queue,))
    producer_process.start()

    # 创建多个消费者进程
    consumer1_process = Process(target=consumer, args=(queue,
lock), name='consumer1')
    consumer1_process.start()

    consumer2_process = Process(target=consumer, args=(queue,
lock), name='consumer2')
    consumer2_process.start()

    consumer3_process = Process(target=consumer, args=(queue,
lock), name='consumer3')
    consumer3_process.start()

    consumer4_process = Process(target=consumer, args=(queue,
lock), name='consumer4')
    consumer4_process.start()

    consumer5_process = Process(target=consumer, args=(queue,
lock), name='consumer5')
    consumer5_process.start()

    # 等待生产者进程完成
    producer_process.join()
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    # 等待所有消费者进程完成
    consumer1_process.join()
    consumer2_process.join()
    consumer3_process.join()
    consumer4_process.join()
    consumer5_process.join()
```

7.1.2 递归锁(可重入锁)

递归锁与互斥锁最大的不同就是它允许同一个进程多次获取同一把锁，这意味着如果一个进程获取了锁，它还可以再次获取锁而不会导致死锁。但是该锁的内部有一个计数器，每当一个进程获取到锁时，计数器就会增加，当进程释放锁时，计数器就会减少。只有当计数器为0时，锁才会真正释放，才会允许其他的进程去获取锁，其他的使用和互斥锁一模一样。

```
from multiprocessing import RLock, Queue, Process, current_process
import time


# 定义生产者函数，它会不断地向队列中添加数据
def producer(queue):
    while True:
        # 循环5次，每次向队列中添加'hello'
        for i in range(6):
            queue.put('hello')
        # 生产者进程休眠2秒
        time.sleep(2)


# 定义消费者函数，它会不断地从队列中取出数据
def consumer(queue, lock):
    while True:
        # 获取锁，确保在消费数据时不会被其他消费者进程打断，并且在执行完毕后自动释放锁
        with lock:
            # 消费者进程休眠2秒
            # time.sleep(2)
            # 检查队列是否为空
            if not queue.empty():
                # 如果队列不为空，取出一个数据
```

华清远见|元宇宙实验室 yyzlab.com.cn

华清远见 HQYJ.COM

```
                res = queue.get()
                # 打印取出的数据
                print(f'{current_process().name}: {res}')


# 主程序入口
if __name__ == '__main__':
    # 创建一个队列，用于生产者和消费者之间传递数据
    queue = Queue(10)
    # 创建一个互斥锁，用于保护队列
    lock = RLock()

    # 创建生产者进程
    producer_process = Process(target=producer, args=(queue,))
    producer_process.start()

    # 创建多个消费者进程
    consumer1_process = Process(target=consumer, args=(queue,
lock), name='consumer1')
    consumer1_process.start()

    consumer2_process = Process(target=consumer, args=(queue,
lock), name='consumer2')
    consumer2_process.start()

    consumer3_process = Process(target=consumer, args=(queue,
lock), name='consumer3')
    consumer3_process.start()

    consumer4_process = Process(target=consumer, args=(queue,
lock), name='consumer4')
    consumer4_process.start()

    consumer5_process = Process(target=consumer, args=(queue,
lock), name='consumer5')
    consumer5_process.start()

    # 等待生产者进程完成
    producer_process.join()

    # 等待所有消费者进程完成
    consumer1_process.join()
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    consumer2_process.join()
    consumer3_process.join()
    consumer4_process.join()
    consumer5_process.join()
```

递归锁的应用场景通常涉及递归函数或递归操作，这些操作需要在同一时间点多次进入临界区。递归锁允许同一个进程多次进入临界区，而不会导致死锁。以下是一些常见的递归锁应用场景：

1. **递归函数**：
   - 递归函数可能会在递归过程中多次进入相同的代码块。如果没有递归锁，这些进入可能会相互阻塞，导致死锁。
2. **递归操作**：
   - 例如，在文件系统的遍历操作中，递归函数可能会在递归过程中多次访问同一目录。
3. **递归数据库操作**：
   - 在数据库中执行递归查询时，可能需要多次访问同一数据集。
4. **递归网络操作**：
   - 在递归处理网络请求时，可能需要多次访问同一服务器或同一数据。
5. **递归图形处理**：
   - 在图形处理或图像处理中，递归算法可能会在递归过程中多次访问同一像素或同一图像区域。
6. **递归算法**：
   - 在算法实现中，递归锁可以用于确保递归算法在递归过程中不会因为锁的竞争而失败。
7. **递归数据结构访问**：
   - 在递归访问数据结构（如树或图）时，递归锁可以确保不会因为递归调用而出现死锁。

华清远见元宇宙实验中心 yyzlab.com.cn

7.2 信号量

信号量是一个更高级的同步机制，它内部维护一个计数器，用于控制对共享资源的最大并发访问数量。在`multiprocessing`模块中，`Semaphore`对象用于此类同步。

- `multiprocessing.Semaphore(value=1)`：创建一个信号量对象，`value`参数指定了初始可用的数量，默认为1。

信号量的方法：

华清远见 HQYJ.COM

- `acquire([timeout=None])`：尝试获取信号量。如果信号量可用，则其值减一并立即返回 `True`。如果信号量不可用，则阻塞直到超时或信号量变为可用。如果没有指定`timeout`或`timeout`为`None`，则一直等待直至信号量可用。
- `release()`：释放一个信号量，其值加一。如果信号量之前已被阻塞，则会唤醒一个正在等待的进程。

```
from multiprocessing import Semaphore, Queue, Process
import time
import sys


def producer(queue):
    while True:
        queue.put('hello')
        time.sleep(0.3)


def consumer1(queue, semaphore):
    while True:
        semaphore.acquire()
        if not queue.empty():
            res = queue.get()
            print(f'consumer1: {res}')
            sys.stdout.flush()
        semaphore.release()


def consumer2(queue, semaphore):
    while True:
        semaphore.acquire()
        if not queue.empty():
            res = queue.get()
            print(f'consumer2: {res}')
            sys.stdout.flush()
        semaphore.release()


def consumer3(queue, semaphore):
    while True:
        semaphore.acquire()
        if not queue.empty():
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
            res = queue.get()
            print(f'consumer3: {res}')
            sys.stdout.flush()
        semaphore.release()


def consumer4(queue, semaphore):
    while True:
        semaphore.acquire()
        if not queue.empty():
            res = queue.get()
            print(f'consumer4: {res}')
            sys.stdout.flush()
        semaphore.release()


def consumer5(queue, semaphore):
    while True:
        semaphore.acquire()
        if not queue.empty():
            res = queue.get()
            print(f'consumer5: {res}')
            sys.stdout.flush()
        semaphore.release()


if __name__ == '__main__':
    queue = Queue(10)  # 增加队列的大小
    semaphore = Semaphore(1)
    producer_process = Process(target=producer, args=(queue,))
    producer_process.start()

    consumer1_process = Process(target=consumer1, args=(queue,
semaphore))
    consumer1_process.start()

    consumer2_process = Process(target=consumer2, args=(queue,
semaphore))
    consumer2_process.start()

    consumer3_process = Process(target=consumer3, args=(queue,
semaphore))
```

华清远见云宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    consumer3_process.start()

    consumer4_process = Process(target=consumer4, args=(queue, semaphore))
    consumer4_process.start()

    consumer5_process = Process(target=consumer5, args=(queue, semaphore))
    consumer5_process.start()

    producer_process.join()
    consumer1_process.join()
    consumer2_process.join()
    consumer3_process.join()
    consumer4_process.join()
    consumer5_process.join()
```

在实际应用中，可以通过调整信号量的初始值来适应不同的并发需求。

既然我们已经熟悉了信号量，那么现在就需要了解其背后的核心机制----PV操作！！！

PV操作是进程同步中的一种基本机制，也被称为信号量机制。PV操作是由荷兰计算机科学家Edsger Dijkstra所提出的，用于解决进程间的同步问题，特别是为了解决临界区问题（即多个进程试图访问共享资源时的竞争问题）。

PV操作的基本概念

- **P操作**（测试和等待操作，有时也称为`wait`操作）：该操作会检查信号量的值。如果信号量的值大于零，则将其减一；如果信号量的值为零，则该进程会被挂起，直到信号量的值变为正数。
- **V操作**（信号操作，有时也称为`signal`或`post`操作）：该操作会增加信号量的值。如果信号量的值增加后仍然小于等于零，则会唤醒一个因P操作而被挂起的进程。

PV操作通常用于解决以下几种问题：

1. **互斥**：确保一次只有一个进程能够进入临界区。例如，多个进程共享一个文件，只允许一个进程读写文件。
2. **同步**：协调两个或多个进程按照某种顺序执行。例如，生产者-消费者问题中，生产者进程生成数据放入缓冲区，消费者进程从缓冲区取出数据处理。
3. **死锁避免**：通过合理设计PV操作序列，可以避免出现死锁情况。

华清远见云计算实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

7.3 事件

事件是一种简单的同步机制，允许一个进程通知一个或多个等待的进程某些事件已经发生，也就是发送一个信号，而其他进程可以根据这个信号做出反应。Event对象的使用场景：

- 一个进程等待另一个进程完成某项任务。
- 控制多个进程间的简单通信。
- 实现对共享资源的访问控制。

在Python中使用`multiprocessing.Event`创建事件对象，其基本方法有：

1. `is_set()`：返回事件是否已设置的状态，如果被设置则返回`True`，否则返回`False`。
2. `set()`：将事件设置为真状态，即`True`，表示可以唤醒正在等待该事件的所有进程。
3. `clear()`：将事件设置为假状态，即`False`，表示没有进程会被唤醒。
4. `wait([timeout])`：阻塞当前进程直到事件被设置为真状态或超时（如果提供了`timeout`参数）。如果没有设置超时时间，则会一直等待直到事件被设置。

事件的使用分为两种情况：

1. 事件只被唤醒一次，然后开始进行自己的工作。

```
from multiprocessing import Event, Process
import time


def producer(start_event, ):
    start_event.set()  # 设置事件以通知消费者可以开始消费
    while True:
        time.sleep(1)


def consumer1(start_event, ):
    start_event.wait()  # 等待事件被设置
    while True:
        print(1)


def consumer2(start_event, ):
    start_event.wait()  # 等待事件被设置
```

华清远见嵌入式实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    while True:
        print(2)



def consumer3(start_event, ):
    start_event.wait()  # 等待事件被设置
    while True:
        print(3)


def consumer4(start_event, ):
    start_event.wait()  # 等待事件被设置
    while True:
        print(4)


def consumer5(start_event, ):
    start_event.wait()  # 等待事件被设置
    while True:
        print(5)


if __name__ == '__main__':
    start_event = Event()

    producer_process = Process(target=producer, args=(start_event,
))
    producer_process.start()

    consumer1_process = Process(target=consumer1, args=
(start_event, ))
    consumer1_process.start()

    consumer2_process = Process(target=consumer2, args=
(start_event, ))
    consumer2_process.start()

    consumer3_process = Process(target=consumer3, args=
(start_event, ))
    consumer3_process.start()
```

华清远见元宇宙实验中心 yyzlab.com.cn

```
    consumer4_process = Process(target=consumer4, args=
(start_event, ))
    consumer4_process.start()

    consumer5_process = Process(target=consumer5, args=
(start_event, ))
    consumer5_process.start()

    producer_process.join()
    consumer1_process.join()
    consumer2_process.join()
    consumer3_process.join()
    consumer4_process.join()
    consumer5_process.join()
```

2. 工作的执行时刻都需要事件来控制。

```
from multiprocessing import Event, Queue, Process
import time


def producer(start_event, ):
    while True:
        start_event.set()  # 设置事件以通知消费者可以开始消费
        time.sleep(0.1)
        start_event.clear()
        time.sleep(3)


def consumer1(start_event, ):
    while True:
        start_event.wait()  # 等待事件被设置
        print(1)


def consumer2(start_event, ):

    while True:
        start_event.wait()  # 等待事件被设置
        print(2)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
def consumer3(start_event, ):

    while True:
        start_event.wait()  # 等待事件被设置
        print(3)


def consumer4(start_event, ):

    while True:
        start_event.wait()  # 等待事件被设置
        print(4)


def consumer5(start_event, ):

    while True:
        start_event.wait()  # 等待事件被设置
        print(5)


if __name__ == '__main__':
    start_event = Event()

    producer_process = Process(target=producer, args=(start_event,
))
    producer_process.start()

    consumer1_process = Process(target=consumer1, args=
(start_event, ))
    consumer1_process.start()

    consumer2_process = Process(target=consumer2, args=
(start_event, ))
    consumer2_process.start()

    consumer3_process = Process(target=consumer3, args=
(start_event, ))
    consumer3_process.start()
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    consumer4_process = Process(target=consumer4, args=
(start_event, ))
    consumer4_process.start()

    consumer5_process = Process(target=consumer5, args=
(start_event, ))
    consumer5_process.start()

    producer_process.join()
    consumer1_process.join()
    consumer2_process.join()
    consumer3_process.join()
    consumer4_process.join()
    consumer5_process.join()
```

7.4 条件变量

条件变量（Condition Variables）是用于协调多个进程的一种机制，它通常与锁一起使用来实现更复杂的同步模式。Python的`multiprocessing`模块提供了`Condition`类，用于支持多进程间的同步。`Condition`对象可以看作是一个锁加上一个或多个条件队列的组合。当一个进程持有锁时，它可以等待特定的条件满足，或者通知其他等待该条件的进程继续执行。

使用multiprocessiong.Condition(lock=None)来创建一个条件变量对象，其中lock可以替换为自己指定的Lock或RLock对象，如果lock为None，则它自己创建一个新的RLock对象并使用。

`Condition`对象的基本方法

1. **acquire()**：获取内部锁。如果无法立即获取锁，则调用者会阻塞直到锁可用。
2. **release()**：释放内部锁。
3. **wait()**：释放内部锁，并使调用进程阻塞，直到接收到通知。当被唤醒时，它会尝试重新获取锁。
4. **wait_for(*predicate*，*timeout=None*)**：释放内部锁，并使调用进程阻塞。*predicate* 应该是一个可调用对象而且它的返回值可被解释为一个布尔值，如果为True，则进程被唤醒，False就继续等待。 *timeout* 参数给出最大等待时间。
5. **notify(n=1)**：唤醒一个或多个正在等待的进程。参数`n`指定了要唤醒的等待进程的数量，默认为1。
6. **notify_all()**：唤醒所有正在等待的进程。

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

7.4.1 使用默认锁

```
from multiprocessing import Condition, Process
import time

def producer(condition):
    while True:
        condition.acquire()
        condition.notify_all()  # 通知所有等待的消费者
        condition.release()
        time.sleep(3)  # 模拟生产者的工作周期


def consumer(condition, number):
    while True:
        condition.acquire()
        print(f'{number}正在等待condition')
        condition.wait()  # 等待条件被满足
        print(f'{number}已释放condition')
        # 证明condition自带的是RLock，而不是Lock
        # condition.release()
        # 这里如果是使用默认的RLock的话，如果不去release，不会导致只有一个进程一直去执行，原因如下：
        '''
        因为在Condition中，如果锁是RLock的话，wait在释放锁时并不是使用的release，而是使用RLock的内部接口，
        即使递归锁被多次获取也可以直接去令计数器归零直接解锁，这样就可以让其他的进程去获取到这个锁，所以如果Condition
        的锁是RLock的话，即使在wait下面没有release也不会出现只有一个进程不断获取锁的情况。如果是Lock的话，
        在wait下面没有release就会导致死锁。
        '''



if __name__ == '__main__':
    condition = Condition()

    producer_process = Process(target=producer, args=(condition,))
    producer_process.start()
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    consumers = [
        Process(target=consumer, args=(condition, 1)),
        Process(target=consumer, args=(condition, 2)),
        Process(target=consumer, args=(condition, 3)),
        Process(target=consumer, args=(condition, 4)),
        Process(target=consumer, args=(condition, 5))
    ]

    for c in consumers:
        c.start()

    producer_process.join()
    for c in consumers:
        c.join()
```

7.4.2 使用with语句

```
from multiprocessing import Condition, Process
import time

def producer(condition):
    while True:
        with condition:
            condition.notify_all()  # 通知所有等待的消费者
            time.sleep(3)  # 模拟生产者的工作周期


def consumer(condition, number):
    while True:
        with condition:
            print(f'{number}正在等待condition')
            condition.wait()  # 等待条件被满足
            print(f'{number}已释放condition')


if __name__ == '__main__':
    condition = Condition()

    producer_process = Process(target=producer, args=(condition,))
    producer_process.start()
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    consumers = [
        Process(target=consumer, args=(condition, 1)),
        Process(target=consumer, args=(condition, 2)),
        Process(target=consumer, args=(condition, 3)),
        Process(target=consumer, args=(condition, 4)),
        Process(target=consumer, args=(condition, 5))
    ]

    for c in consumers:
        c.start()

    producer_process.join()
    for c in consumers:
        c.join()
```

7.4.3 使用自定义锁

在大多数情况下，不需要额外的锁。`Condition` 对象内部的锁已经足够来管理条件变量的操作。

```
from multiprocessing import Condition, Lock, Process
import time


def producer(condition):
    while True:
        with condition:
            condition.notify_all()  # 通知所有等待的消费者
        time.sleep(3)  # 模拟生产者的工作周期


def consumer(condition, number):
    while True:
        with condition:
            print(f'{number}抢到了锁，正在等待condition')
            condition.wait()  # 等待条件被满足
            print(f'condition已经触发，{number}释放了锁，')


if __name__ == '__main__':
    lock = Lock()
    condition = Condition(lock=lock)
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    producer_process = Process(target=producer, args=(condition,))
    producer_process.start()

    consumers = [
        Process(target=consumer, args=(condition, 1)),
        Process(target=consumer, args=(condition, 2)),
        Process(target=consumer, args=(condition, 3)),
        Process(target=consumer, args=(condition, 4)),
        Process(target=consumer, args=(condition, 5))
    ]

    for c in consumers:
        c.start()

    producer_process.join()
    for c in consumers:
        c.join()
```

8. 进程池

进程池是一组预先创建的空闲进程，它们等待执行任务，主进程负责将任务分配给进程池中的空闲进程去执行。进程池可以管理进程的创建和销毁，避免了频繁地创建和销毁进程带来的开销，通过进程池可以轻松的实现多任务的并行处理。

8.1 为什么使用进程池

- **效率**：相比于手动创建和管理多个进程，使用进程池可以更高效地利用系统资源。
- **简化**：进程池简化了并行编程的复杂性，开发者不需要关注进程的创建和销毁细节。
- **控制**：可以限制同时运行的进程数量，防止系统资源被过度消耗。

8.2 进程池的创建

在Python中，进程池的创建有两种方式。

华清远见云宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

8.2.1 使用multiprocessing库

在Python中，使用multiprocessing 模块中的Pool创建进程池：

```
import multiprocessing.Pool
multiprocessing.Pool(processes=None, initializer=None, initargs=
(), maxtasksperchild=None)
```

- **processes**：进程池中的进程数。如果`processes`为`None`，则默认使用系统的处理器核心数。
- **initializer**：每个工作进程启动时要执行的可调用对象，默认为None。如果是None，则调用initializer(*initargs)。
- **initargs**：传递给`initializer`的可变参数元组。
- **maxtasksperchild**：工作进程退出之前可以完成的任务数，完成后用一个新的工作进程来替代原进程，来让闲置的资源被释放。maxtasksperchild默认是None，意味着只要Pool存在工作进程就会一直存活。

返回对象的主要方法为：

- **apply(func, args=())**：在一个池工作进程中执行func(args,*kwargs),然后返回结果。需要强调的是:此操作并不会在所有池工作进程中并执行func函数。如果要通过不同参数并发地执行func函数，必须从不同线程调用p.apply()函数或者使用p.apply_async。它是阻塞的。apply很少使用。
- **apply_async(func, args=(), kwds={}, callback=None)**：异步地执行函数`func`。`args`是传递给`func`的位置参数元组，`kwds`是传递给`func`的关键字参数字典。`callback`是一个回调函数，当`func`执行完成后会被调用。
- **map(func, iterable, chunksize=None)**：将`iterable`中的每个元素作为参数传递给`func`，并返回结果列表。此方法类似于内置函数`map`，但是它是并行的。
- **map_async(func, iterable, chunksize=None, callback=None)**：与`map`类似，但是是异步的。`chunksize`指定每次分配给进程的迭代器元素数量。
- **imap(func, iterable, chunksize)**：imap 与 map的区别是，map是当所有的进程都已经执行完了，再返回结果，imap()则是立即返回一个iterable可迭代对象。
- **imap_unordered(func, iterable, chunksize)**：与imap相同，只不过不保证返回的结果顺序与进程添加的顺序一致。
- **close()**：阻止任何新的任务被提交到池中。一旦所有任务完成，工作进程会退出。
- **terminate()**：立即终止所有工作进程，不再处理未处理的任务。
- **join()**：等待所有工作进程退出。必须在`close()`或`terminate()`之后调用。

华清远见嵌入式学习中心 www.fanyi.com.cn

华清远见 HQYJ.COM

8.2.1.1 apply函数和apply_async函数

同步操作:

在同步操作中，调用者必须等待直到被调用的操作完成才能继续执行下一步。简单来说，就是按照顺序一步一步地执行，每一步都必须等待前一步完成后才能开始。例如，在同步函数调用中，主调函数会等待被调用的函数返回结果后才会继续执行下面的代码。

特点:

- 顺序执行。
- 阻塞等待。
- 通常更易于理解和实现。
- 可能导致性能瓶颈，特别是在需要等待长时间操作时。

异步操作:

在异步操作中，调用者不需要等待被调用的操作完成就可以继续执行其他任务。当被调用的操作完成时，会通过回调函数、事件通知等方式告知调用者。这种方式允许程序同时处理多个任务，提高了效率和响应速度。

特点:

- 并行或多任务执行。
- 不阻塞等待。
- 可以提高系统的整体吞吐量。
- 实现相对复杂，需要处理回调或其他机制来管理非阻塞操作。

apply函数:

```
import time
from multiprocessing import Pool


def factorial(n):
    while True:
        if n <= 1:
            break
        print(n)
        n -= 1
        time.sleep(1)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
def test1():
    print(123)


def test2():
    print(456)


if __name__ == "__main__":
    with Pool(processes=4) as pool:  # 使用单个进程进行同步计算
        result = pool.apply(factorial, (5,))  # 计算5的阶乘
        pool.apply(test1)
        pool.apply(test2)
```

apply_async函数:

```
from multiprocessing import Pool

# 定义计算斐波那契数列的函数
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)

# 定义一个回调函数，用于处理计算结果
def handle_result(result):
    print(f"Fibonacci result: {result}")

if __name__ == "__main__":
    # 创建一个进程池
    with Pool(processes=4) as pool:
        # 定义要计算的斐波那契数列的索引列表
        fib_indices = [35, 36, 37, 38]  # 这些数字较大，计算可能需要一
些时间

        # 使用apply_async异步计算斐波那契数列的值，并设置回调函数
        results = []
        for index in fib_indices:
            result = pool.apply_async(fibonacci, (index,),
callback=handle_result)
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
            results.append(result)

        # # 等待所有异步任务完成
        # for result in results:
        #     result.get()  # 获取结果，这里主要是为了等待任务完成
        pool.close()
        pool.join()
    print("All Fibonacci calculations are complete.")
```

方法apply_async()和map_async()的返回值都具有以下的方法：

1. get(timeout)：用于获取执行结果。如果timeout不是None且在timeout秒内仍然没有执行完得到结果，就会抛出异常。
2. wait(timeout)：阻塞，直到返回结果，或在timeout秒后超时。
3. ready()：返回执行状态，是否已经完成任务。
4. succesful()：判断调用是否已经完成并且未引发异常。如果还没获得结果就抛出异常。

8.2.1.2 map函数和map_async函数

map函数：

```
from multiprocessing import Pool

def square(x):
    """计算给定数字的平方"""
    return x * x

if __name__ == "__main__":
    # 创建一个进程池
    with Pool(processes=4) as pool:
        # 定义要计算的数字列表
        numbers = [1, 2, 3, 4, 5]

        # 使用map同步计算每个数字的平方
        results = pool.map(square, numbers)

    print("Results:", results)
```

map_async函数：

```
import time
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
from multiprocessing import Pool

def square(x):
    """计算给定数字的平方"""
    return x * x

if __name__ == "__main__":
    # 创建一个进程池
    with Pool(processes=4) as pool:
        # 定义要计算的数字列表
        numbers = [1, 2, 3, 4, 5]

        # 使用map_async异步计算每个数字的平方
        async_result = pool.map_async(square, numbers)

        # 主进程可以继续执行其他任务
        print("Main process continues to run in parallel.")

        # 等待所有子进程完成
        pool.close()
        pool.join()

        # 获取异步计算的结果
        results = async_result.get()

    print("Results:", results)
```

8.2.2 使用concurrent.futures库

在Python中，使用concurrent.futures的ProcessPoolExecutor创建进程池。

```python
concurrent.futures.ProcessPoolExecutor(max_workers=None,
mp_context=None,
                                       initializer=None,
initargs=())
```

- `max_workers`：指定进程池中可以同时运行的最大进程数。如果设置为 `None` 或未指定，则默认为机器的处理器数量，最多为61。
- `mp_context`：指定多进程上下文。默认情况下，`ProcessPoolExecutor` 使用 `multiprocessing.get_context()` 来获取上下文。这允许你选择不同的上下

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

文，例如 `spawn`、`fork`、`forkserver` 等，这些上下文可能提供不同的功能，如更好的资源隔离、更好的安全性等。

- `initializer`：一个可选的可调用对象，每个工作进程在启动时都会调用它。这可以用来执行进程的初始化操作，例如设置进程局部存储。
- `initargs`：一个元组，其中包含传递给 `initializer` 的参数。

其主要的方法为：

- `submit(fn, *args, **kwargs)`：提交一个可调用对象 `fn` 到进程池，并返回一个 `Future` 对象，该对象的result方法可以用来获取结果。
- `map(func, *iterables, timeout=None, chunksize=1)`：它允许你将一个函数 `func` 应用于多个可迭代对象 `iterables` 中的元素，并且并行地在多个进程上执行这些函数调用。`timeout`是可选参数，用于设置阻塞等待每个任务完成的最大秒数。如果 `timeout` 被触发，将引发 `concurrent.futures.TimeoutError`。`chunksize`是可选参数，指定每次提交给进程池的任务数量。一个较大的 `chunksize` 可以减少进程间通信的开销，但它也会增加内存消耗，因为它会保存更多的任务结果。返回的结果是一个迭代器。
- `shutdown(wait=True)`：等待所有进程完成当前任务后关闭进程池。如果`wait`参数设置为`True`，进程池会等待所有任务完成；如果设置为`False`，进程池会立即返回，不等待任务完成。使用with语句管理时，with语句结束时会自动调用shutdown。

1. **submit的应用：**

```
from concurrent.futures import ProcessPoolExecutor
import time

# 定义一个简单的函数，用于执行一个计算密集型的任务
def compute_square(n):
    return n * n


if __name__ == '__main__':

    numbers = [1, 2, 3 ,4, 5]
    results = []

    # 使用with语句创建ProcessPoolExecutor实例
    with ProcessPoolExecutor(max_workers=3) as executor:
```

华清远见 yyzlab.com.cn

华清远见 HQYJ.COM

```
        for num in numbers:
            # 提交计算任务到进程池
            future = executor.submit(compute_square, num)
            results.append(future)

        for res in results:
            print(res.result())
```

2. **map的应用：**

```
from concurrent.futures import ProcessPoolExecutor
import time


# 定义一个简单的函数，计算一个数字的平方
def square(number):
    return number * number


if __name__ == '__main__':
    # 数字列表
    numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

    # 使用with语句创建ProcessPoolExecutor实例
    with ProcessPoolExecutor(max_workers=3) as executor:
        # 使用map方法并行执行square
        results = executor.map(square, numbers)

    # 输出结果
    for result in results:
        print(result)
```

华清远见元宇宙实验中心 yyzlab.com.cn

需要注意的是：

进程池之间进程的通信及同步需要使用的库是multiprocessing.Manager()下的队列、锁等内容。

8.3. 手动创建与进程池的比较

8.3.1 手动创建多进程

优点：

1. **灵活性**：手动创建进程可以更精确地控制每个进程的创建和销毁，可以针对特定任务定制进程的行为。
2. **直接控制**：可以直接与进程对象交互，例如，可以设置进程的名称、守护状态等。
3. **简单的并行结构**：对于简单的并行任务，手动创建进程可以更加直观。

缺点：

1. **资源管理**：需要手动管理进程的生命周期，包括创建、销毁和异常处理。
2. **开销大**：进程的创建和销毁开销较大，如果频繁创建和销毁大量进程，可能会导致性能问题。
3. **同步问题**：需要手动处理进程间同步（例如，使用锁、信号量等）。

8.3.2 进程池创建多进程

优点：

1. **高效管理**：进程池会管理进程的生命周期，包括进程的创建和销毁，减少了开销。
2. **资源限制**：可以限制同时运行的进程数量，防止系统资源被过度消耗。
3. **简化并行**：简化了并行任务的处理，不需要手动创建和销毁进程。
4. **任务分发**：可以使用`apply`，`apply_async`，`map`，`map_async`等函数方便地分发任务到进程池。

缺点：

1. **灵活性较低**：与手动创建进程相比，进程池提供的是一种更通用的解决方案，可能不适合所有特定场景。
2. **可能造成阻塞**：如果进程池中的所有进程都在忙，而又有新的任务需要执行，那么这些任务可能会被阻塞，直到有进程可用。

华清远见元宇宙实验中心 yyzlab.com.cn